

App Development II Project Report

1. STUDENT DETAILS

Name: Dheepika R

Roll Number: 24f2004166

Email: 24f2004166@ds.study.iitm.ac.in

About Me: A passionate developer focused on building efficient, full-stack web applications that solve real-world organizational challenges through intuitive design and robust backend logic.

2. PROJECT DETAILS

Project Title: Ascentia - Placement Portal Application V2

Problem Statement:

Traditional campus placement processes often suffer from fragmented communication between students, companies, and administrators. Manual tracking of job drives, application statuses, and student eligibility leads to delays and data inconsistency.

Approach:

The project follows a Role-Based Access Control (RBAC) architecture to provide separate functionalities for Administrators, Companies, and Students. The backend is developed using Flask and exposes RESTful APIs for application logic, authentication, and data management. SQLAlchemy ORM with SQLite is used for programmatic database creation and relational data persistence, while Vue.js 3, Bootstrap 5, and custom CSS are used to build a responsive single-page frontend.

The system centralizes the placement workflow by enabling Administrators to verify companies and placement drives, Companies to create and manage recruitment drives and applications, and Students to maintain profiles, search and apply for opportunities, and track application and placement status. Additional components such as JWT-based authentication, Redis caching, Celery-based background processing, Chart.js analytics, and an ATS-style resume screening module enhance the system's security, performance, automation, and usability.

3. AI/LLM DECLARATION

I used LLM to assist in writing SQLAlchemy model definitions, creating professional UI components with custom CSS and improving variable naming consistency. The extent of AI/LLM usage is around <10%, limited to **code suggestions and documentation formatting**. All final implementation logic, debugging, and integration were done manually.

4. TECHNOLOGIES AND FRAMEWORKS USED

TECHNOLOGY / FRAMEWORK	PURPOSE
Flask	Backend API development and application logic
Vue.js 3	Reactive single-page application frontend
SQLite	Local relational database storage
SQLAlchemy	Object Relational Mapping for managing SQLite database
JWT	Token-based authentication and role-based access control.
Bootstrap 5	Provides responsive layouts, UI components, and styling.
Redis	API caching and task message brokering
Celery	Executes asynchronous and scheduled background tasks
Chart.js	Placement analytics and data visualization
JavaScript, HTML5 & CSS3	Frontend logic, styling, validation, API interaction.
Jinja2	HTML template rendering and frontend integration

5. DATABASE SCHEMA

Tables:

1. users Table

Serves as the central authentication and user management table. It stores user credentials and access information, including id, email, password, role, and is_active.

2. student_profiles Table

Stores academic and professional information of registered students. Key attributes include user_id, full_name, student_id, branch, year, cgpa, skills, resume_link, placement_status, and profile_pic.

3. company_profiles Table

Maintains information about registered recruiting organizations. It contains details such as user_id, company_name_full, industry, location, website, contact_person, phone, and approval_status.

4. job_drives Table

Stores information related to company placement drives. Important fields include company_id, company_name, job_title, description, skills_required, salary_range, eligibility_criteria, application_deadline, status, drive_status, and created_at.

5. applications Table

Maintains student applications submitted for placement drives. It records student_id, drive_id, status, applied_at, interview_date, and feedback.

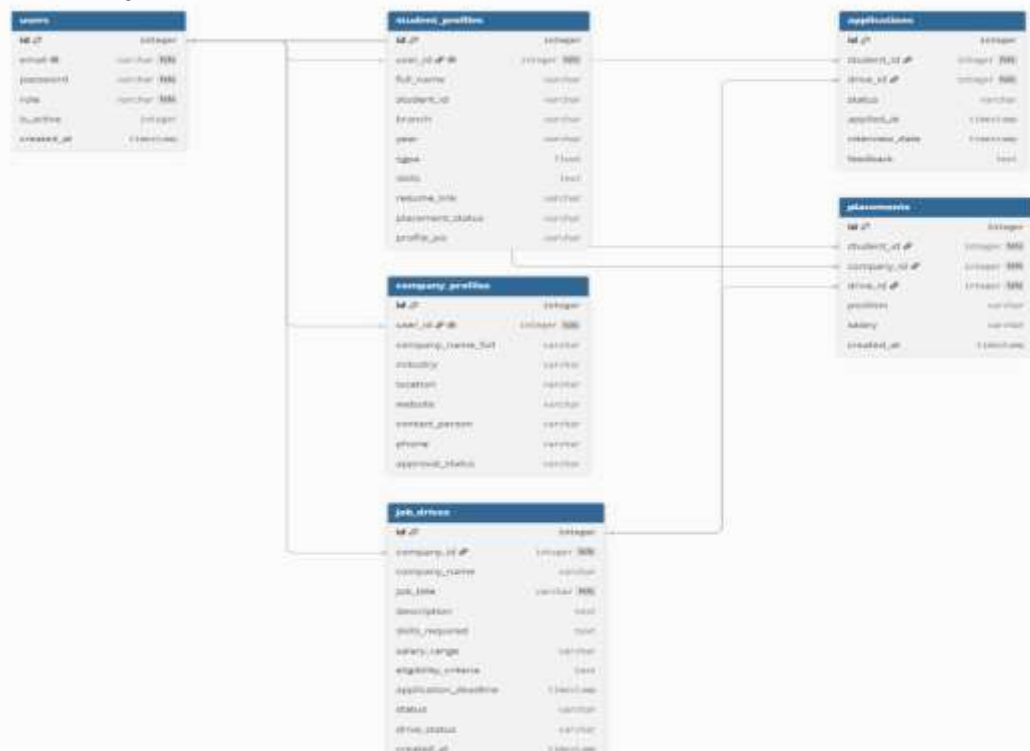
6. placements Table

Stores details of confirmed student placements, including student_id, company_id, drive_id, position, salary, and created_at.

Entity Relationships

The major relationships between the database entities are:

- **One-to-One:** users → student_profiles
- **One-to-One:** users → company_profiles
- **One-to-Many:** company_profiles → job_drives
- **One-to-Many:** student_profiles → applications
- **One-to-Many:** job_drives → applications
- **Application-Based Relationship:** student_profiles ↔ job_drives through applications
- **Placement Relationship:** applications → placements



6. API RESOURCE ENDPOINTS

YAML API Definition File:

Included separately in the submission ZIP as api.yaml.

ENDPOINT	METHOD	DESCRIPTION
/api/auth/login	POST	Authenticates users and issues a signed JWT token.
/api/auth/register	POST	Registers new student or company accounts.
/api/admin/stats	GET	Retrieves aggregated statistics for the Admin Dashboard.
/api/admin/companies/<id>/approve	POST	Approves a registered company account.
/api/admin/drives/<id>/approve	POST	Approves a company placement drive.
/api/company/drives	POST	Creates a new placement drive for administrative approval.
/api/company/drives/<id>/export-csv	POST	Initiates an asynchronous CSV export of candidate applications.
/api/student/apply/<id>	POST	Submits a student application for a placement drive.
/api/public/stats	GET	Retrieves aggregated public placement statistics and skill-demand data.

7. ARCHITECTURE AND FEATURES

The application follows the **Model-View-Controller (MVC)** pattern a Vue.js 3 frontend and Flask REST API backend. SQLAlchemy ORM is used to interact with the SQLite database. JWT-based authentication provides secure, stateless authentication and role-based access control for Admin, Company, and Student users. Redis is used for caching and Celery handles asynchronous background tasks.

Implemented Features

- **Admin Dashboard:** Provides centralized management of students, companies, placement drives, applications, and platform statistics. Administrators can approve companies and drives.
- **Company Dashboard:** Enables companies to create and manage placement drives, view applicants, update application statuses, schedule interviews, and manage recruitment activities.
- **Student Dashboard:** Allows students to manage their profiles, search approved placement drives, apply for jobs, and track application and placement status.

- **Authentication & Role-Based Access:** Implements JWT-based authentication with separate access controls and dashboards for Admin, Company, and Student roles.
- **Application & Placement Tracking:** Maintains application history and tracks candidates through stages such as Applied, Shortlisted, Interview, Selected, Rejected, and Placed.
- **Redis Caching:** Caches frequently accessed data such as placement drives, dashboard statistics, and company information to improve API response performance, with cache invalidation.
- **Asynchronous Background Tasks:** Uses Celery with Redis for tasks such as CSV exports, interview reminders, and placement report generation.
- **Analytics & ATS:** Provides placement analytics.
- **Responsive UI:** Uses Vue.js, Bootstrap 5, and custom CSS to provide a consistent and responsive interface across the application.

8. VIDEO PRESENTATION

Drive Link: https://drive.google.com/file/d/1ysv85S48iZaTtrkTv_7st-rzwuQGvO3/view?usp=sharing